Solution: Valid Perfect Square

Problem Statement

Given a positive integer `num`, return `true` if `num` is a perfect square or `false` otherwise.

A **perfect square** is an integer that is the square of an integer. In other words, it is the product of some integer with itself.

You must not use any built-in library function, such as `sqrt`.

Examples

1. - **Input:** 49
   - **Expected Output:** true
   - **Justification:** (7 * 7) equals 49.
2. - **Input:** 55
   - **Expected Output:** false
   - **Justification:** There is no integer whose square is 55.
3. - **Input:** 0
   - **Expected Output:** false
   - **Justification:** 0 is not considered a perfect square.

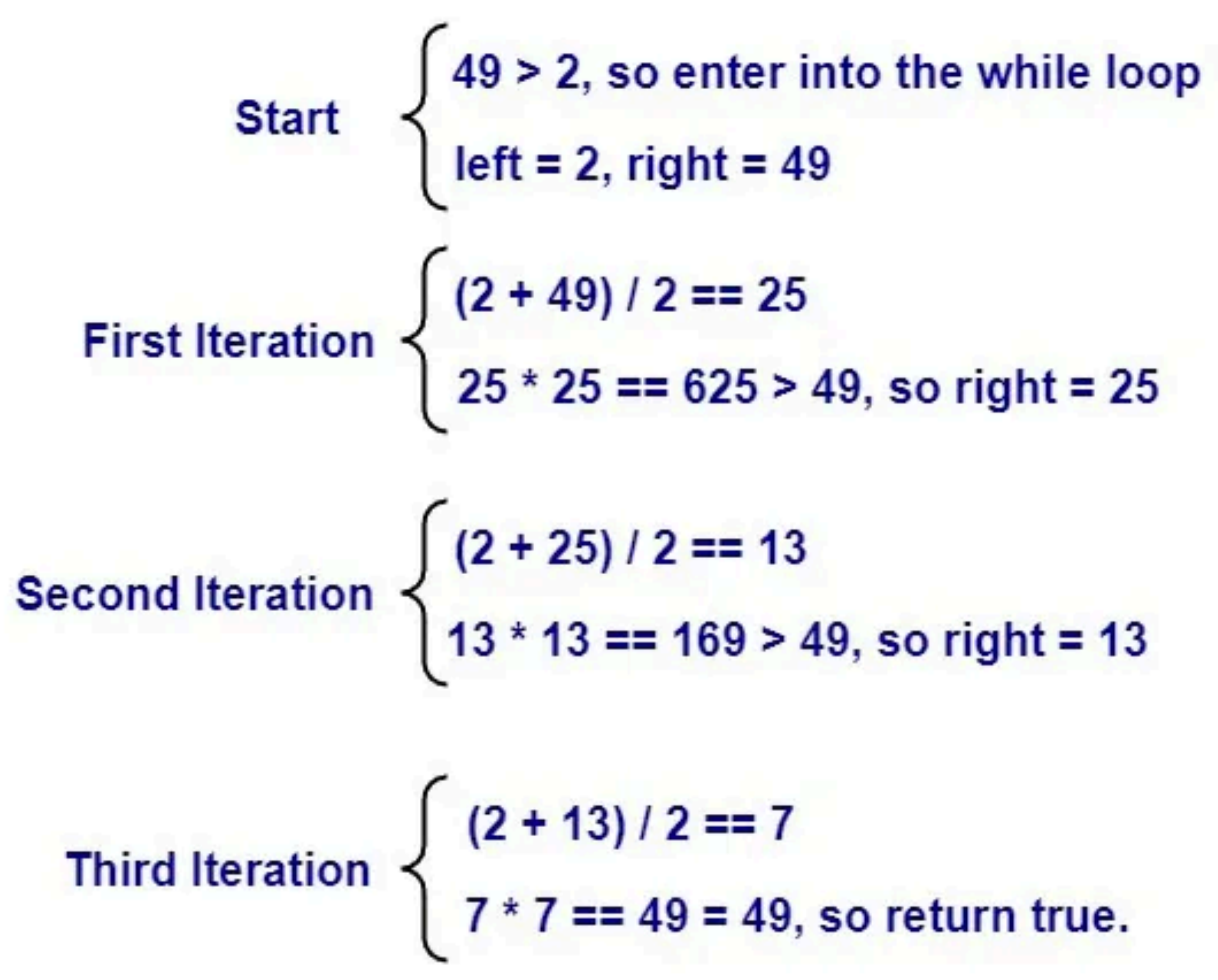
Solution

- **Step 1: Start**
  - Start by checking if the number is less than 2.
- **Step 2: Binary Search Approach**
  - Implement a binary search between 2 and the number.
  - Calculate the middle and check if its square is equal to the given number.
  - If it is equal, return true.
  - If it's less, perform a binary search on the right half.
  - If it's more, perform a binary search on the left half.

Algorithm Walkthrough

Given an input of 49:

- Start by checking if 49 is less than 2. It is not, so proceed.
- Implement a binary search between 2 and 49.
  - Calculate the middle number: ((2 + 49) / 2 = 25)
  - (25 * 25) is more than 49, so perform a binary search on the left half (2 to 25).
  - New middle: ((2 + 25) / 2 = 13)
  - (13 * 13) is more than 49, so perform a binary search on the left half (2 to 13).
  - New middle: ((2 + 13) / 2 = 7)
  - (7 * 7) equals 49, return true.



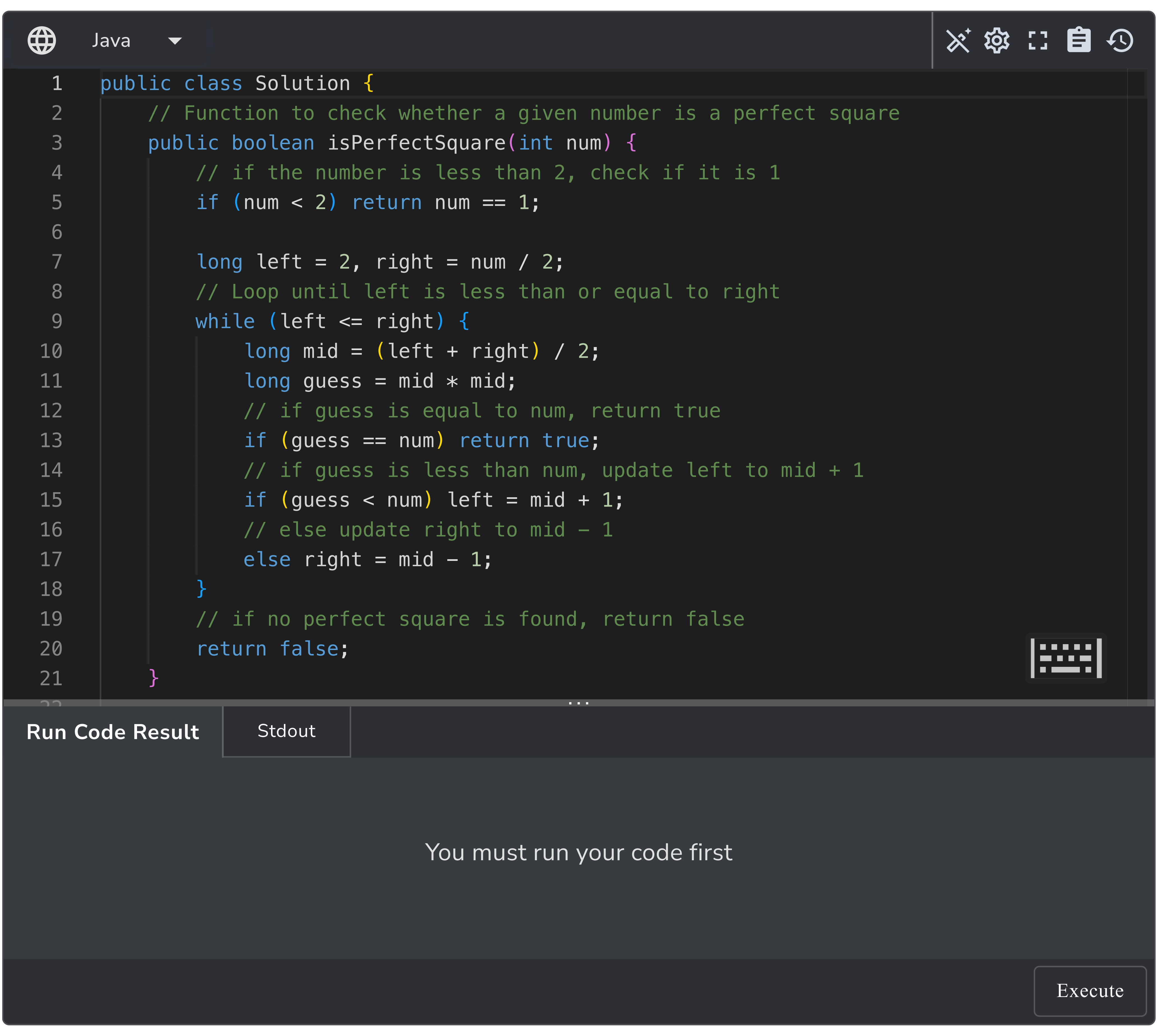
Start
49 > 2, so enter into the while loop
left = 2, right = 49

First Iteration
(2 + 49) / 2 == 25
25 * 25 == 625 > 49, so right = 25

Second Iteration
(2 + 25) / 2 == 13
13 * 13 == 169 > 49, so right = 13

Third Iteration
(2 + 13) / 2 == 7
7 * 7 == 49 = 49, so return true.

Code



```java
public class Solution {
    // Function to check whether a given number is a perfect square
    public boolean isPerfectSquare(int num) {
        // if the number is less than 2, check if it is 1
        if (num < 2) return num == 1;

        long left = 2, right = num / 2;
        // Loop until left is less than or equal to right
        while (left <= right) {
            long mid = (left + right) / 2;
            long guess = mid * mid;
            // if guess is equal to num, return true
            if (guess == num) return true;
            // if guess is less than num, update left to mid + 1
            if (guess < num) left = mid + 1;
            // else update right to mid - 1
            else right = mid - 1;
        }
        // if no perfect square is found, return false
        return false;
    }
```

You must run your code first

Complexity Analysis

- **Time Complexity**: The time complexity of this algorithm is (O(log n)) because it employs a binary search strategy, effectively halving the search space in each iteration.
- **Space Complexity**: The space complexity is (O(1)), as it uses a constant amount of additional memory space, not dependent on the input size.

← Previous
Valid Perfect Square (Easy)

Next →
Best Time To Buy And Sell (Easy)

☐ Mark as Completed